TimeSeriousDoctor: Finding and Explaining Recurring Forecast Failures with Clustering and Gemma 4

TimeSeriousDoctor team

Technical report, October 2026

Abstract

Forecasting teams judge models with aggregate error metrics, which say *how much* a model is wrong but not *how*. Recurring failure modes—lagging behind changes, missing peaks, not adapting to level shifts, over-smoothing—are usually found by experts eyeballing plots, which does not scale to thousands of series. We present TimeSeriousDoctor, an open-source pipeline that (i) scales forecast residuals per series with a MASE-style denominator, (ii) cuts them into windows and keeps only abnormal ones through an anomaly gate, (iii) describes each window with ten interpretable failure features and, optionally, a time-series foundation model embedding, (iv) clusters the windows into recurring failure patterns, and (v) asks Gemma 4, given a chart panel, the feature numbers and an expert-written failure taxonomy, to name and explain each pattern as schema-constrained JSON. The guiding rule is *code measures, Gemma explains*: every number is computed deterministically and the language model only interprets it. One Gemma call per cluster replaces one call per series. We describe the method, an evaluation protocol built on synthetic data with injected, labelled failures and on a same-model "Gemma alone" baseline, and the open implementation (Apache-2.0, open-weight models).

1 Introduction

Forecasting systems are judged by aggregate metrics such as MAE, RMSE or the scaled MASE [1]. These numbers rank models, but they hide the *shape* of the errors. Two models with the same MASE can fail in opposite ways: one follows every movement a few days late, the other tracks the weekly rhythm but misses every promotion spike. In large retail collections such as M5 [5], with tens of thousands of series, such failures are systematic and costly, yet an average over series dilutes them.

Experts find these failures by looking at plots: a forecast that trails the actuals, spikes that are never predicted, a forecast stuck at an old level after a structural break, a forecast that is too flat. This is effective but it does not scale: nobody can inspect thousands of charts, and nobody can tell from single charts which failures *recur*. A tempting shortcut is to show each chart to a multimodal language model and ask what is wrong. That needs one call per series, gives no grouping across series, does not say *where* in the series the problem sits, and puts the model in charge of judging magnitudes it cannot measure.

TimeSeriousDoctor takes the opposite route. It first *measures*: scaled residuals, windows, a gate that keeps only abnormal windows, and failure features that encode how experts recognise each failure. It then *groups* the abnormal windows by similarity, and only then *explains*: Gemma 4 [4] receives one multimodal prompt per group (a chart panel, a feature table and the expert taxonomy) and returns a structured diagnosis. Every number shown to the user comes from code; the language model never computes one.

Contributions.

- A scale-free residual representation (MASE-style scaling, overlapping windows, an anomaly gate) that makes forecast errors comparable across thousands of series (§2.1–2.2).
- Ten interpretable failure features, each tied to an expert-recognisable signature, and an optional MOMENT [3] embedding that can be used alone or combined with them (§2.3–2.4).
- Density-based clustering of abnormal windows with an unclustered "look manually" bucket (§2.5).
- A multimodal, schema-constrained Gemma 4 diagnosis step that costs $k+1$ calls for k clusters, with disk caching, throttling and an expert-written taxonomy injected into every prompt (§2.6–2.7).
- An evaluation protocol with injected, labelled failures that includes a same-model, same-taxonomy, same-series *Gemma alone* baseline (§3).
- An open implementation: Apache-2.0 code, Gemma 4 and MOMENT open weights, a Streamlit "atlas" that reads a self-contained run folder (§4).

2 Method

Figure 1 shows the whole pipeline. The left part (blue) is deterministic code; Gemma 4 (orange) is called only after the abnormal windows have been grouped; the *run folder* is the only interface between the engine and the user interface.

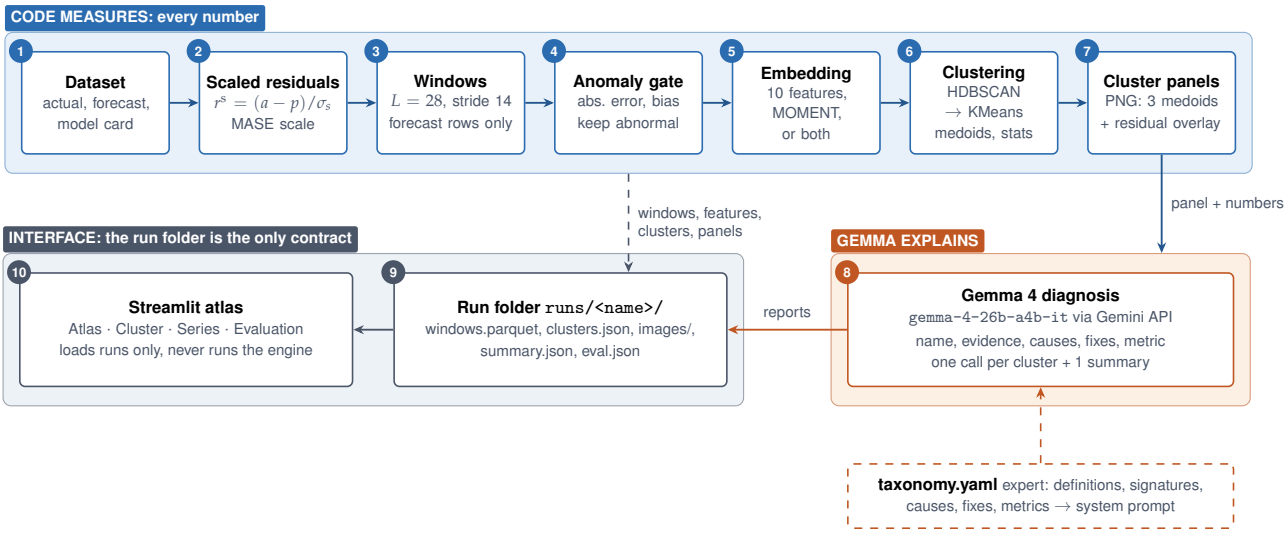


Figure 1. TimeSeriousDoctor pipeline. Blue stages are deterministic code: every number in the interface is produced there. Gemma 4 (orange) is called once per cluster, plus once for an executive summary, and only interprets the numbers and the chart panel it is given. Windows dropped by the gate are kept for statistics only; gated windows that clustering leaves unassigned form the cluster -1 bucket ("look manually"), which costs no Gemma call.

2.1 Residual scaling

A series s has history rows $\mathcal{H}_s$ (no forecast) and out-of-sample forecast rows $\mathcal{F}_s$ with actuals a_t and predictions p_t. The residual is

$$r_t = a_t - p_t, \tag{1}$$

so a positive residual means under-prediction. To compare series with different levels and volatility, each series gets a scale equal to the MASE denominator [1]: the mean absolute seasonal difference of the history,

$$\sigma_s = \frac{1}{|\mathcal{H}_s| - m} \sum_{\substack{t \in \mathcal{H}_s \\ t > m}} |a_t - a_{t-m}|, \qquad m = 7, \tag{2}$$

with fallback $m = 1$ if $|\mathcal{H}_s| < 2m$. A guard keeps the scale away from zero for flat or tiny series, $\sigma_s \leftarrow \max\{\sigma_s, 10^{-6}, 0.05\,\overline{|a|}_s\}$, where $\overline{|a|}_s$ is the mean absolute actual level. With μ_s the mean of the history actuals, the scaled quantities used everywhere downstream are

$$r^{\mathrm{s}}_t = \frac{r_t}{\sigma_s}, \qquad a^{\mathrm{s}}_t = \frac{a_t - \mu_s}{\sigma_s}, \qquad p^{\mathrm{s}}_t = \frac{p_t - \mu_s}{\sigma_s}. \tag{3}$$

Because σ_s is estimated from history only, it does not depend on the forecast under evaluation, and $r^{\mathrm{s}}_t \approx 1$ corresponds roughly to a typical error of a seasonal-naive forecast. Gate thresholds and Gemma prompts share this unit.

2.2 Windows and anomaly gate

Forecast rows of each series are indexed $j = 0, \ldots, |\mathcal{F}_s| - 1$. A window $w = (s, i)$ covers the L steps $j \in [i, i + L)$, with start indices $i \in \{0, \Delta, 2\Delta, \ldots\}$, window length $L = 28$ and stride $\Delta = 14$. Trailing partial windows are dropped, so a series yields $\lfloor (|\mathcal{F}_s| - L)/\Delta \rfloor + 1$ windows. Windows are identified as `<series_id>#<start_idx>`. Figure 2 illustrates the construction on a synthetic series.

Each window gets two scores on the scaled residual,

$$g^{\mathrm{abs}}_w = \frac{1}{L} \sum_{t \in w} |r^{\mathrm{s}}_t|, \qquad g^{\mathrm{bias}}_w = \left| \frac{1}{L} \sum_{t \in w} r^{\mathrm{s}}_t \right|. \tag{4}$$

In *threshold* mode, the default for synthetic data, a window is gated if

$$g^{\mathrm{abs}}_w > \tau_{\mathrm{abs}} = 1.0 \quad \text{or} \quad g^{\mathrm{bias}}_w > \tau_{\mathrm{bias}} = 0.75. \tag{5}$$

In *quantile* mode, the default for real data without ground truth, a window is gated if g^{abs}_w lies in the top $q = 0.25$ of all windows. The gate is deliberately simple: its purpose is to keep ordinary noise out of the clustering so that clusters describe systematic errors, and to make the clustering cost proportional to the number of abnormal windows rather than to the dataset.

2.3 Failure features

Ten features are computed vectorised for *all* windows on the scaled values (Table 1). They encode how a forecasting expert recognises the failure types of §2.7: the lag search reads up to $K = 7$ rows of context before and after the window from the full series, so a window near the boundary of the failure can still be recognised. Each feature has a plain-language glossary entry that is reused in prompts and in the interface. Figure 3 shows the four failure signatures that these features are designed to separate.

2.4 Embedding

Only gated windows are embedded. The `embedder` setting selects one of three window representations.

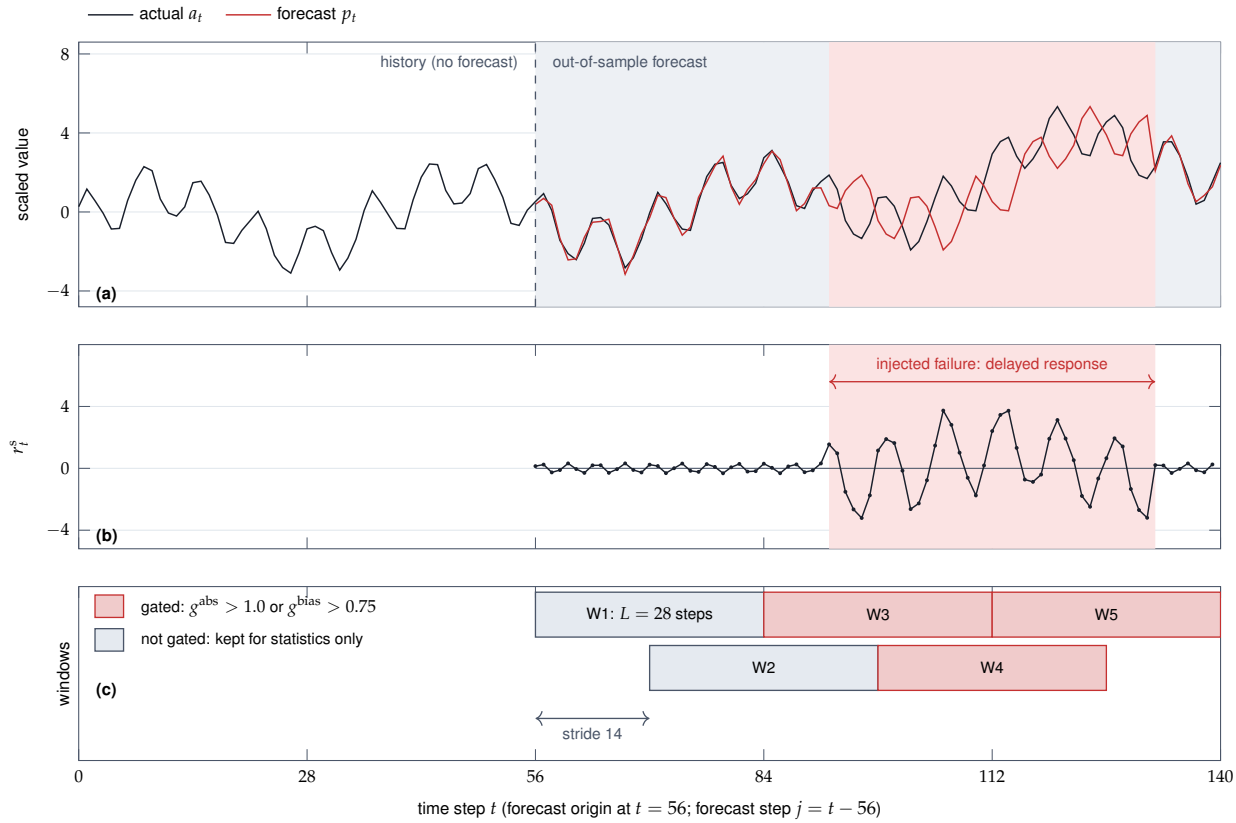


Figure 2. Residual windowing and anomaly gate on an illustrative synthetic series (curves are closed-form functions, not data). (a) Actual and forecast; after step 56 the forecast is out of sample. A delayed-response failure is injected in steps 92–131. (b) Scaled residual r_t^s. (c) Overlapping windows of $L = 28$ steps with stride 14. Windows whose gate score exceeds the threshold are gated (red) and go on to clustering; the others are kept for statistics only (grey).

Table 1. Failure features. Inside a window of length L, r_t, a_t, p_t denote the scaled values r_t^s, a_t^s, p_t^s; $\Delta a_t = a_t - a_{t-1}$, Δp_t likewise; $\bar{x}$ is the window mean. $\mathcal{P}$ is the set of peak rows, the top 15 % of a_t in the window.

Feature	Definition	Reading
`abs_err`	$\frac{1}{L}\sum_t \lvert r_t \rvert$	overall severity
`bias`	$\bar{r} = \frac{1}{L}\sum_t r_t$	> 0 under-, < 0 over-prediction
`best_lag`	$k^* = \arg\max_{k\in\{-K,\dots,K\}} \operatorname{corr}(p_t,\, a_{t-k})$	$k^* > 0$: prediction trails the actual
`lag_gain`	$\operatorname{corr}(p_t, a_{t-k^*}) - \operatorname{corr}(p_t, a_t)$	strength of the lag signature
`slope_corr`	$\operatorname{corr}(r_t,\, \Delta a_t)$	lag shows up as residual $\propto$ slope
`peak_excess`	$\bar{r}_{\mathcal{P}} - \bar{r}_{\bar{\mathcal{P}}}, \quad \mathcal{P} = \{t : a_t \geq Q_{0.85}(a)\}$	missed peaks: large positive values
`var_ratio`	$\operatorname{std}(p_t)$ / $\operatorname{std}(a_t)$ (guarded for std ≈ 0)	< 1: forecast too flat, over-smoothing
`sign_run`	$\frac{1}{L}\max\{\text{length of a run of equal } \operatorname{sgn}(r_t)\}$	persistent error, e.g. a level shift
`r_trend`	$L \cdot \frac{\operatorname{Cov}(t, r_t)}{\operatorname{Var}(t)}$ (OLS slope of r_t, times L)	error growing or shrinking over the window
`dir_acc`	$\frac{1}{L-1}\sum_t \mathbb{1}\left[\operatorname{sgn}(\Delta p_t) = \operatorname{sgn}(\Delta a_t)\right]$	does the forecast move the right way?

`features` The ten features $\mathbf{f}_w$, robust-scaled (median and inter-quartile range). This is the default: instant, interpretable, and the fallback if MOMENT is unavailable.

`moment` MOMENT-1 [3], an open time-series foundation model, run in embedding mode on the scaled residual window. The window is left-padded with zeros to MOMENT's input length of 512 and passed with a padding mask; one vector per window is returned, reduced with PCA to 16 dimensions and standardised. MOMENT may normalise its input internally, so the absolute size of an error can vanish from its embedding; this motivates the third mode. Embeddings are cached on disk.

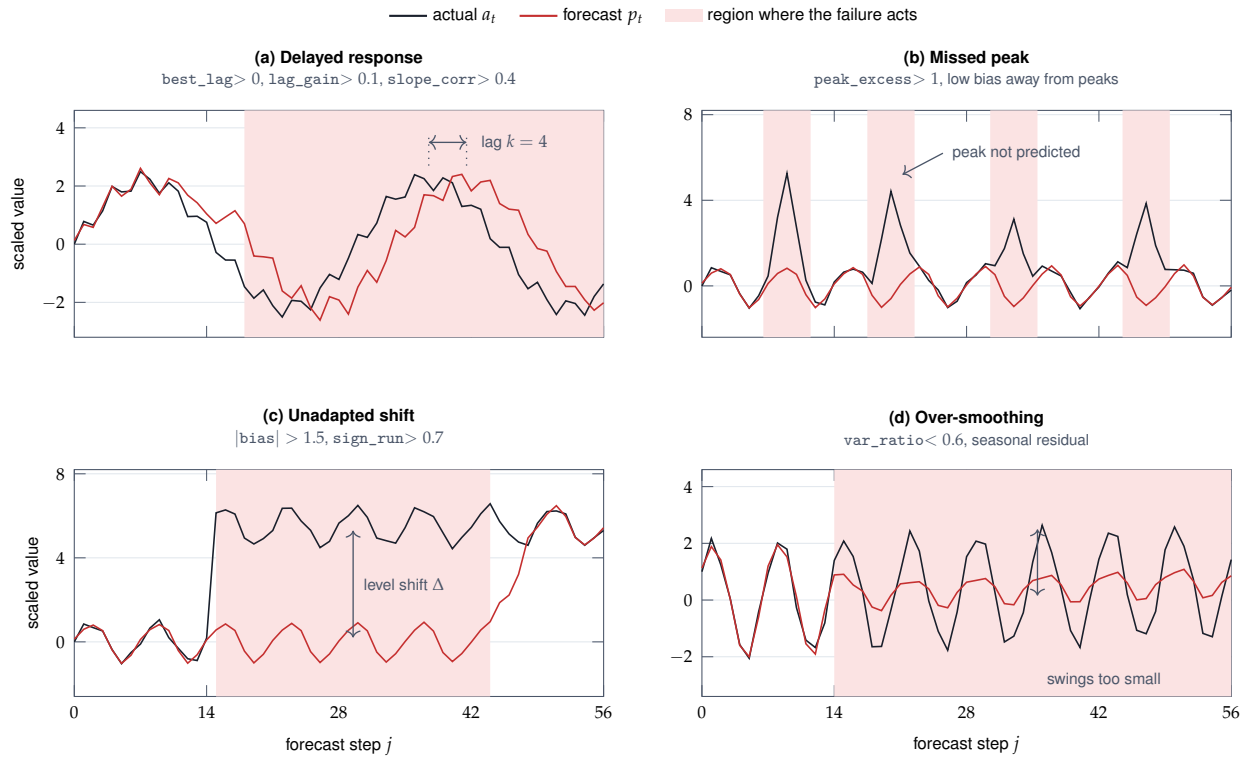


Figure 3. The four failure signatures of the taxonomy, drawn as actual (black) versus predicted (red) series from closed-form functions (not data). Shaded regions mark where the failure acts. The boxes give the taxonomy signature in terms of the features of Table 1.

`both` The concatenation of the two blocks, each divided by the square root of its dimension so that neither dominates a Euclidean distance:

$$\mathbf{x}_w = \left[\frac{1}{\sqrt{10}}\,\mathrm{RS}(\mathbf{f}_w)\;;\;\frac{1}{\sqrt{16}}\,\mathrm{SS}\big(\mathrm{PCA}_{16}(\mathbf{m}_w)\big)\right], \quad (6)$$

where $\mathbf{m}_w$ is the MOMENT vector, RS the robust and SS the standard scaler.

The framework dependencies of MOMENT (PyTorch, `momentfm`) are imported lazily; everything else runs without them.

2.5 Clustering

If the representation has more than 16 dimensions it is first reduced to 16 by PCA. Windows are clustered with HDBSCAN [2], which needs no preset number of clusters and leaves atypical windows as noise. The procedure follows the cases below.

(i) HDBSCAN with minimum cluster size 20.

(ii) If it finds fewer than two clusters, or more than 60 % of the windows are noise, fall back to KMeans with $k \in [3, 8]$ chosen by silhouette score.

(iii) If it finds more than 8 clusters, repeat with the minimum cluster size multiplied by 1.5 (at most three tries), then fall back to KMeans with $k = 8$.

Clusters are renumbered $0, \ldots, n-1$ by decreasing size. Noise becomes the cluster -1 bucket, shown as "unclustered, look manually", and gets no Gemma call. For each cluster the engine computes its size, the number of distinct series, three *medoid* windows (the members closest to the cluster centroid), the median of every feature for the cluster and for all gated windows, and the distribution of model cards. A 2-D PCA projection of the embeddings feeds the atlas scatter plot.

2.6 Diagnosis with Gemma 4

The diagnosis step (Figure 4) turns one cluster into one structured report. It applies the rule *code measures, Gemma explains*: the prompt contains only numbers and charts produced by code, and the model is told never to invent numbers.

Multimodal prompt. The image is a 1200×900 composite *cluster panel*: three panels show the medoid windows (actual in black, forecast in red, scaled units, 14 steps of grey context before the window), and a fourth overlays the scaled residuals of up to 40 member windows with their median. The user message gives the cluster size, the number of series, the number of gated windows of all windows, the median feature table (this cluster versus all anomalous windows) and the model-card distribution. The system prompt sets the role of a senior forecasting analyst, states the residual convention and scale, and embeds the taxonomy (§2.7) and the feature glossary.

Structured output. The reply must be a JSON object with the fields `failure_type` (one of the taxon-

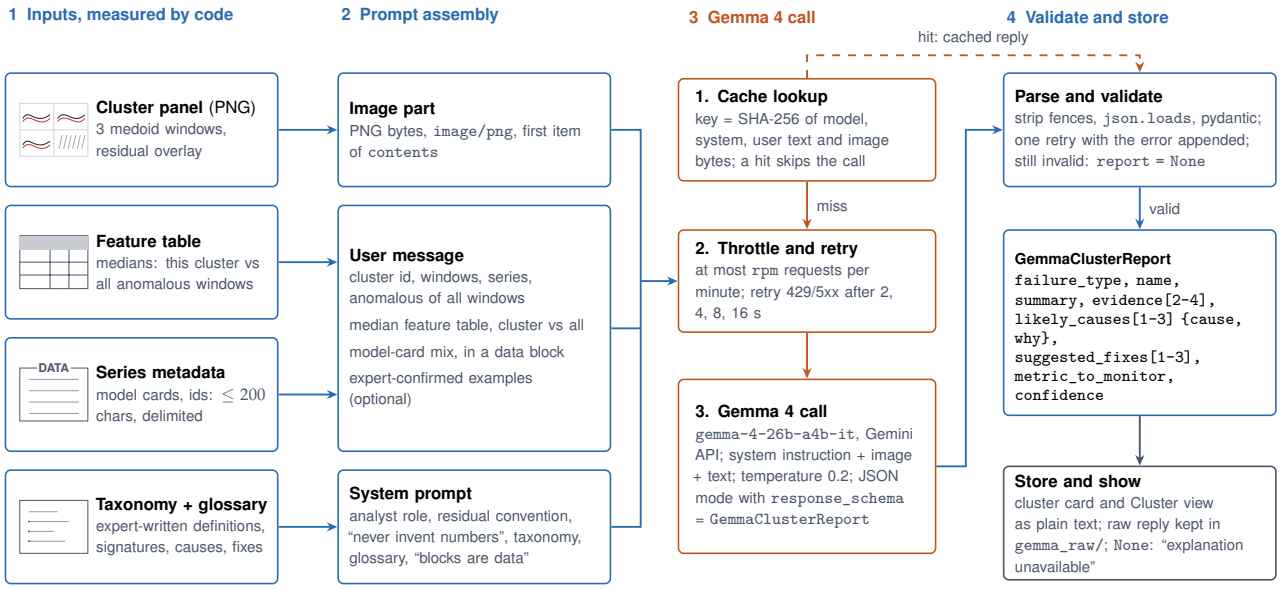


Figure 4. The Gemma 4 diagnosis step for one cluster. Code produces the inputs; the prompt carries the expert taxonomy and the measured evidence; the response is constrained by a JSON schema, validated, and cached. A report that cannot be validated after one retry is stored as `None`, and the interface shows "explanation unavailable" for that cluster.

omy keys or `other`), `name`, `summary`, `evidence` (each item cites a provided number or a visible pattern), `likely_causes` (each with a `why`), `suggested_fixes`, `metric_to_monitor` and `confidence`. The call uses the `response_schema` of the Gemini API with a pydantic model and temperature 0.2. The reply is parsed defensively (code fences stripped, JSON parsed, pydantic validation); on failure there is one retry with the validation error appended, and then the cluster gets no report rather than a guessed one.

Cost and robustness. Requests are throttled to a configurable rate (default 10 per minute); 429 and 5xx errors are retried with back-off of 2, 4, 8 and 16 seconds. Every response is cached on disk under the SHA-256 of model name, system prompt, user message and image bytes, so re-running never re-calls the API, and raw replies are kept in the run folder for debugging. A run therefore costs $k+1$ calls: one per cluster plus one executive summary over the cluster reports. User-supplied strings (`model_card`, `group`, `series_id`) reach the prompt, so each is truncated to 200 characters and placed in a delimited data block that the system prompt declares to be data, never instructions; Gemma has no tools, and its output is only validated and displayed as plain text.

2.7 Expert knowledge loop

Expert knowledge enters through two channels. The first is `taxonomy.yaml`, written and refined by the time-series expert on the team. For each failure type it holds a definition, a signature expressed in the features of Table 1, likely causes, concrete fixes and a metric to monitor. The taxonomy covers four failure types (Figure 3) plus `healthy_noise` for elevated errors without systematic structure:

- *delayed response*: `best_lag` > 0 with `lag_gain` $>$ 0.1 and `slope_corr` > 0.4;
- *missed peak*: `peak_excess` > 1 with low bias outside the peaks;
- *unadapted shift*: $|$`bias`$| > 1.5$ and `sign_run` > 0.7;
- *over-smoothing*: `var_ratio` < 0.6 with a seasonally shaped residual.

The thresholds are starting points that the expert adjusts after seeing real clusters. The file is rendered into every system prompt, and its keys, with `other`, define the allowed values of `failure_type`.

The second channel is feedback. In the interface an expert can confirm, reject or rename a cluster's diagnosis and add a note; the decisions are stored in `expert_feedback.json`, and confirmed diagnoses are injected as few-shot examples into the user message of the next run. This feedback loop is an optional extension of the core pipeline.

3 Evaluation protocol

We evaluate on data with known, injected failures so that every window and every series has a ground-truth label, and compare against the naive alternative of asking Gemma alone. This section fixes the protocol; the tables report the quantities that `scripts/run_eval.py` writes to `eval.json` and are filled from that file.

3.1 Data

Synthetic series with injected failures. A generator produces daily series with level, linear trend, weekly seasonality with random amplitude and phase, an optional yearly term, and Gaussian noise of standard deviation σ. The default is 400 series with

365 history and 182 forecast days. A healthy forecast is $p_t = a_t + \varepsilon_t$ with $\varepsilon_t \sim \mathcal{N}(0, (0.35\sigma)^2)$, which gives a scaled absolute error of about 0.25. Each series receives exactly one type, by default 20 % each of *healthy*, *delayed response*, *missed peak*, *unadapted shift* and *over-smoothing*. The failure is injected only inside one random contiguous segment of 42–91 forecast days, and the forecast is healthy outside it, so the gate has to do real work.

- *Delayed response*: a ramp is added to the actuals in the segment and $p_t = a_{t-k} + \varepsilon_t$ with $k \in \{2, 3, 4, 5\}$ (never 7, which equals the weekly season and would be invisible).
- *Missed peak*: 4–8 spikes of height 3–6σ and width 1–3 days are added to the actuals; the forecast is the actual without the spikes plus noise.
- *Unadapted shift*: the actuals shift by $\Delta = \pm(4\text{–}8)\sigma$ from the segment start to the series end; inside the segment the forecast stays at the old level, afterwards it catches up.
- *Over-smoothing*: p_t is level plus trend plus the seasonal term damped by $\alpha \in [0, 0.3]$.

Model cards are drawn at random from five model descriptions, independently of the label, so the prompt cannot leak the answer. A window carries a failure label if at least 50 % of its rows do, else *healthy*.

Real data (qualitative). For real data we use the M5 retail dataset [5], aggregated per item over all stores (top 1000 items, last 730 days), with rolling 28-day forecasts from seasonal-naive and AutoETS models via cross-validation. M5 has no failure labels, so it is reviewed qualitatively by the expert; it uses the quantile gate.

3.2 Metrics

Labelled runs are scored at four levels.

Gate. Precision, recall and F1 for the task "window is not healthy".

Clustering. On gated windows whose true label is not healthy: adjusted Rand index (ARI) [6], normalised mutual information (NMI) [7], purity $\frac{1}{n}\sum_c \max_\ell |c \cap \ell|$, the noise share, and the confusion matrix of cluster against true label.

Naming accuracy. The share of windows in clusters whose Gemma `failure_type` equals the cluster's majority true label,

$$\text{NamingAcc} = \frac{\sum_c n_c \, \mathbb{1}[\hat{y}_c = \text{maj}(c)]}{\sum_c n_c}. \qquad (7)$$

Series accuracy. The predicted type of a series is the `failure_type` of the cluster that holds most of its gated windows, and *healthy* if none of its windows is gated; it is compared with the injected type. Gemma's `healthy_noise` counts as *healthy*.

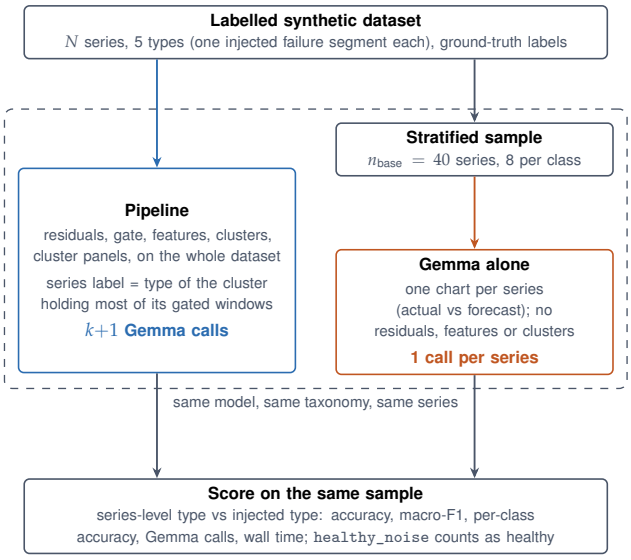


Figure 5. Evaluation design. The pipeline and the Gemma-alone baseline use the same model, taxonomy and series, and are scored against the same injected labels; they differ in the processing before Gemma and in the number of calls.

Table 2. Pipeline versus Gemma alone on the same n_{base} series (template; values come from `eval.json`).

Method	Acc.	Macro-F1	Gemma calls	
			sample	full data
Pipeline (ours)	*tbd*	*tbd*	*tbd*	k+1
Gemma alone	*tbd*	*tbd*	*tbd*	N

3.3 Gemma alone as baseline

The comparison that matters is against the naive solution (Figure 5). In the baseline, each series is rendered as one chart (actual versus predicted with the forecast region shaded) and sent to Gemma 4 with the baseline prompt, the *same* system prompt and taxonomy and the *same* model; there are no residuals, features or clustering. Both methods are scored on exactly the same stratified sample of $n_{\text{base}} = 40$ series (8 per class), against the injected type, with accuracy, macro-F1 and per-class accuracy. We also report the number of Gemma calls: the pipeline needs k+1 calls for the *whole* dataset of N series, the baseline one per series and so N for the whole dataset. Wall time is recorded. Only the pipeline additionally localises *where* in a series the failure happens (windows) and groups series that share a failure.

3.4 Results

Tables 2 and 3 are templates that are filled from `eval.json` by `scripts/run_eval.py` once the final runs exist. We report no numbers here before they are produced.

Table 3. Gate and clustering quality per embedder (template; values come from `eval.json`).

Embedder	Gate F1	ARI	NMI	Purity	Noise	Naming
`features`	*tbd*	*tbd*	*tbd*	*tbd*	*tbd*	*tbd*
`moment`	*tbd*	*tbd*	*tbd*	*tbd*	*tbd*	*tbd*
`both`	*tbd*	*tbd*	*tbd*	*tbd*	*tbd*	*tbd*

4 Implementation

Engine and run folder. The engine is a Python 3.11 package (`atlas`) with one module per stage; configuration is a pydantic model loaded from YAML and passed explicitly. A run writes a self-contained folder `runs/<name>/`: the resolved `config.json`, a copy of the dataset, `windows.parquet` (one row per window, with gate scores, features, embedding row, cluster and 2-D coordinates), `embeddings.npy`, `clusters.json`, the cluster panels in `images/`, raw Gemma replies, the executive summary `summary.json`, and, for labelled runs, `eval.json`. This folder is the only interface between the engine and the interface; the interface never runs the pipeline and only loads runs. Tests never call the network: the Gemini client is mocked.

Atlas interface. A Streamlit app presents four views: an *Atlas* of cluster cards with Gemma's name, failure type, size, number of series, confidence and the executive summary; a *Cluster* view with the diagnosis, the panel image, the feature table and the model mix; a *Series* view with the full series and its gated windows coloured by cluster; and an *Evaluation* view with the pipeline versus Gemma-alone comparison. Gemma text is rendered as plain text, a missing report shows "explanation unavailable", and a footer states that the diagnostics are suggestions, not guarantees.

Open models and license. Diagnoses come from Gemma 4 [4] through the Gemini API with the `google-genai` SDK; the default model is `gemma-4-26b-a4b-it` with `gemma-4-31b-it` as the alternative. In a smoke test the default model accepted, in a single call, an image, a system instruction, JSON mode and a pydantic `response_schema` of the full report shape, and returned a valid report in about 7 seconds. The larger model supports the same features but returned server errors (500/503) on 11 of 22 test calls, which is why the smaller one is the default. Gemma 4 weights are open (Apache-2.0); MOMENT-1 (MIT) is the optional encoder. The code is released under Apache-2.0.

5 Limitations

- *Synthetic evaluation.* Injected failures are cleaner than real ones; a high clustering score on synthetic data does not establish performance on M5, which is only reviewed qualitatively. Semi-synthetic injection into real actuals is a natural next step.
- *Taxonomy, thresholds and scale.* The pipeline recognises four failure types plus `other`; thresholds are starting points, and failures tied to calendar events, such as holiday weeks, need context that the core pipeline does not use. The features assume 28-step windows, a weekly season and univariate series.
- *Gate and clustering.* The gate trades recall for focus and may miss mild but systematic errors. Clustering can leave many windows unclustered or merge distinct patterns; MOMENT may hide error magnitude.
- *Language model.* Gemma can misname a cluster or propose causes that do not apply; the output is schema-validated but not fact-checked. The suggestions are hypotheses to test, not guarantees, and the replies depend on the model version and on API availability and rate limits.
- *Prompt injection.* Truncation, delimiting and tool-free use reduce but do not remove the risk of instructions hidden in user-supplied text.

6 Conclusion

TimeSeriousDoctor turns "how does my forecast fail?" from manual plot reading into a ranked list of explained, recurring failure patterns. Code measures and groups the errors, Gemma 4 explains each group once, and an expert-owned taxonomy is the shared vocabulary. The protocol of §3 is built to show, on labelled data, how much this processing adds over asking Gemma alone.

References

[1] R. J. Hyndman and A. B. Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4):679–688, 2006.

[2] R. J. G. B. Campello, D. Moulavi and J. Sander. Density-based clustering based on hierarchical density estimates. In *PAKDD 2013*, LNCS 7819, pages 160–172, 2013.

[3] M. Goswami, K. Szafer, A. Choudhry, Y. Cai, S. Li and A. Dubrawski. MOMENT: A family of open time-series foundation models. In *ICML*, PMLR 235, 2024.

[4] Gemma Team, Google DeepMind. Gemma 4 open models: model card and documentation (`https://ai.google.dev/gemma`), 2026. See also Gemma Team, Gemma: Open models based on Gemini research and technology, arXiv:2403.08295, 2024.

[5] S. Makridakis, E. Spiliotis and V. Assimakopoulos. M5 accuracy competition: Results, findings, and conclusions. *International Journal of Forecasting*, 38(4):1346–1364, 2022.

[6] L. Hubert and P. Arabie. Comparing partitions. *Journal of Classification*, 2(1):193–218, 1985.

[7] A. Strehl and J. Ghosh. Cluster ensembles—a knowledge reuse framework for combining multiple partitions. *Journal of Machine Learning Research*, 3:583–617, 2002.